

Portfolio QA Architecture Master Plan

A structured approach to architecting a QA system featuring QA analysis, testing, engineering, and DevOps CI/CD work, augmented holistically by Generative AI.

1. Project Overview & System Under Test (SUT)

- **SUT:** Portfolio Site (Astro + Tailwind, hosted on GitHub Pages).
- **Key Feature:** Custom telemetry health monitoring system for demo showcases.
- **Core Objective:** Build an 85% working showcase demonstrating holistic Gen AI integration in the QA lifecycle, emphasizing practical, maintainable test management.

2. Generative AI Tool Matrix

Tool	Use Case in Pipeline
ChatGPT (Web UI)	High-level BRS synthesis (reverse engineering Astro code), test case design, logic validation.
GitHub Copilot (VS Code)	Code completion, Page Object Model (POM) boilerplate, locator strategies.
Gemini (CLI / API)	Failure triage, log analysis, auto-generating bug reports in the CI/CD pipeline.

3. The "85% Showcase" Definition of Done (DoD)

To avoid over-engineering and infinite iteration, the project is considered "DONE" when the following criteria are met:

- **BRS:** A baseline Business Requirements Specification is generated, manually refined, and covers the core static pages + telemetry component.
- **Framework:** An automation framework (e.g., Playwright) is running locally using a clean Page Object Model (POM) architecture.
- **AI QA Engineering Demonstrated:**
 - Test data is generated/managed efficiently.
 - Resilient locator strategy (e.g., data-testid) is implemented to avoid brittle tests.
 - At least one synthetic mock for the telemetry system network requests is working.
- **CI/CD:** A GitHub Actions workflow successfully triggers and runs the test suite on push/PR.
- **Smart Triage:** A custom script successfully parses a failed pipeline log and outputs a structured Markdown bug report detailing the root cause.

4. Execution Phases

Phase 1: BRS & Test Design

1. Extract route layouts and Astro components.
2. Prompt ChatGPT to reverse-engineer a BRS with Acceptance Criteria.
3. Manual review and gap filling.
4. Generate core test cases from the BRS.

Phase 2: Framework Implementation

1. Set up the automation framework repository.
2. Implement the POM structure.
3. Write test scripts for core BRS features (using Copilot).
4. Implement synthetic mock data for the telemetry system.

Phase 3: CI/CD & AI Triage

1. Create GitHub Actions YAML for test execution.
2. Introduce an intentional failure to test the triage system.
3. Implement a script to send logs to Gemini and generate a bug report.